

VICA — Visual Chess Analyzer

Technical Analysis Document

Automated Chess Position Analysis from Images | Introduction to Computer Vision — Final Project | EPICODE Institute of Technology

1. Problem Statement

Chess players frequently encounter positions they wish to analyze — in books, during online games, or while observing a match. Evaluating such a position with a chess engine normally requires manually re-entering every piece into an analysis tool, which is slow and error-prone.

VICA (Visual Chess Analyzer) automates this step. Given a digital screenshot of a chess board, the system detects the position, reconstructs it in standard FEN notation, and uses a chess engine to evaluate the position and suggest the best move. The task is an example of **structured recognition**: rather than a single classification, the system must produce 64 coordinated predictions that together form a valid, rule-constrained output.

2. Methodology

VICA is implemented in Python using OpenCV (classical image processing), PyTorch (deep learning), scikit-learn (evaluation metrics), and the Stockfish engine. It is organised as a modular pipeline of six stages, each an independent, testable component.

2.1 Preprocessing and Feature Representation

A board screenshot is divided into an 8×8 grid of 64 square sub-images, each resized to 224×224 pixels and normalised with ImageNet statistics. VICA combines **handcrafted** and **learned** features. Occupancy detection uses handcrafted features — Canny edge density and grayscale standard deviation: an empty square is smooth and uniform, while an occupied square contains a piece with a distinct outline. This fast classical filter ensures the neural network runs only on squares that contain a piece.

2.2 Core Model: Piece Classification

Piece identification uses a **ResNet-18** convolutional neural network with **transfer learning**. The network is initialised with ImageNet-pretrained weights and its final layer replaced with a 6-class head (Pawn, Knight, Bishop, Rook, Queen, King). A **two-stage fine-tuning** strategy is used: first the backbone is frozen and only the classifier head is trained; then the final residual block is unfrozen and trained with a tenfold-reduced learning rate. Lower layers, which extract universal features, stay frozen, while deeper layers adapt to chess pieces. The reduced learning rate prevents catastrophic forgetting. Data augmentation (flips, rotations, brightness and contrast variation) reduces overfitting on a small dataset.

2.3 Post-processing, FEN and Engine Analysis

Because the classifier is probabilistic, a rule-based post-processing stage validates the position against the structural rules of chess — one king per side, piece-count limits, and the constraint that pawns cannot occupy the first or last rank — and corrects impossible configurations. The board is then converted to a FEN string; since the CNN predicts type but not colour, a separate classical step determines colour through brightness analysis of the segmented piece. The FEN is passed to Stockfish via the UCI protocol using MultiPV mode, which returns not only the best move but also the top alternative moves, their evaluations, and the principal variation — the predicted sequence of moves following each candidate. This gives the user a richer tactical understanding rather than a single isolated suggestion. Position legality is validated first, so classification errors are reported gracefully rather than producing misleading output.

2.4 Web Application

As an optional extension, VICA includes a web application built with Streamlit. It reuses all pipeline modules unchanged and adds only a presentation layer: the user uploads a board screenshot, selects the side to move and board orientation, and receives the detected position, FEN, and best move. This demonstrates the benefit of the modular design — the same core logic serves both a command-line and a web interface.

3. Experimental Results

Two datasets were used: a public dataset of about 556 photographs of physical chess pieces, and a synthetic dataset of about 480 images generated to match the digital-screenshot domain. Both were split into training (70%), validation (15%), and test (15%) sets with stratified sampling and a fixed random seed.

The physical-piece model reached **85.39%** test accuracy. Two-stage fine-tuning was decisive: validation accuracy rose from about 60% after stage one to over 85% after unfreezing the final block. Per-class metrics are shown in Table 1.

Class	Precision	Recall	F1-score	Support
Bishop	0.765	0.929	0.839	14
King	0.800	0.667	0.727	12
Knight	1.000	0.941	0.970	17
Pawn	1.000	0.941	0.970	17
Queen	0.615	0.615	0.615	13
Rook	0.882	0.938	0.909	16
Macro average	0.844	0.838	0.838	89

Table 1: Per-class metrics for the physical-piece classifier on the test set.

Knight and Pawn achieve the highest F1-scores (0.970); Queen is the weakest class (0.615), most often confused with Bishop and King, as the confusion matrix in Figure 1 shows. The training curves in Figure 2 show a sharp improvement at the start of stage two.

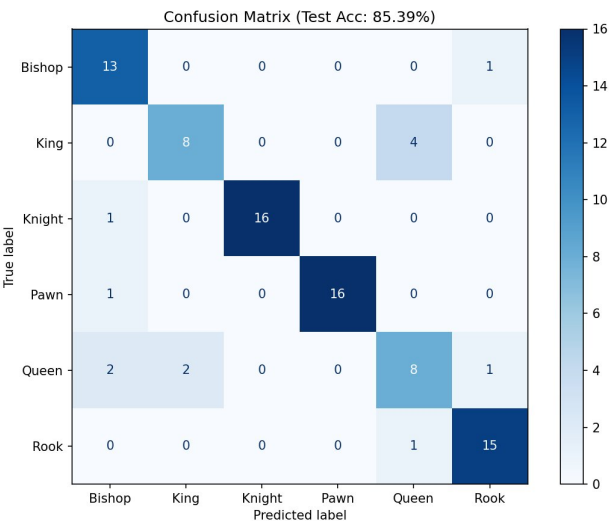


Figure 1: Confusion matrix on the test set (85.39%).

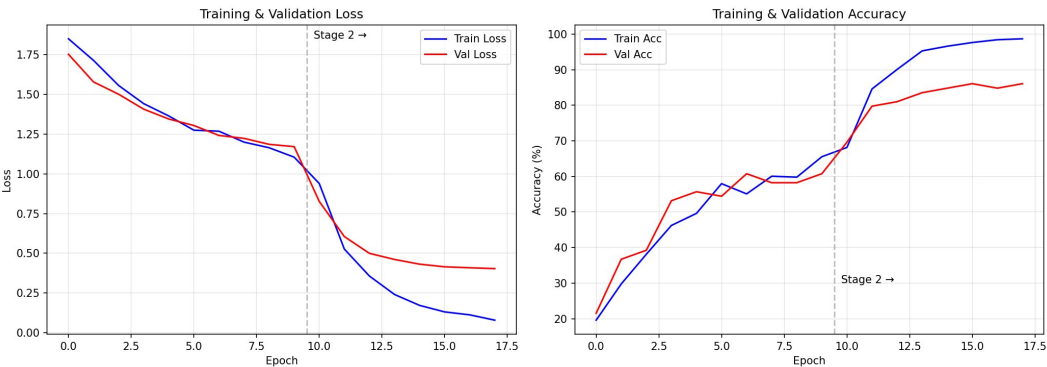


Figure 2: Training and validation curves across the two-stage fine-tuning process.

On clean in-domain screenshots, the full pipeline correctly reconstructs the position, generates a valid FEN, and returns an engine recommendation. On a starting-position screenshot the system detects all 32 pieces and Stockfish returns a standard opening move (Figure 3).

[illegible]

Figure 3: End-to-end pipeline output, from screenshot to best-move recommendation.

4. Failure Analysis

Domain shift. The classifier’s performance is bounded by the diversity of its training data. A model trained on one piece style performs poorly on a different style, and a model trained on digital pieces fails on photographs of physical boards.

Compounding errors. Reconstructing a full board requires 64 simultaneous correct classifications. Even at 95% per-square accuracy, full-board accuracy is 0.95^{64} , only about 4%. A single misclassified square can corrupt the position; this motivates the rule-based post-processing stage.

Colour detection. An early version measured overall square brightness and failed, because a dark piece on a light square contains many bright background pixels. The fix was to segment the piece region first and analyse only its pixels.

Illegal positions. When errors compound, the reconstructed position may be illegal. VICA validates legality and reports the problem instead of passing invalid data to the engine — graceful degradation rather than confident but meaningless output.

The 100% synthetic result. The digital-piece model reached 100% test accuracy, but this is interpreted critically: the synthetic dataset was generated from a small set of source sprites, so training and test samples are nearly identical — a form of data leakage. The result reflects memorisation, not generalisation. It is acceptable for the specific use case of standard digital screenshots, but the 85% physical-piece model is a more honest measure of generalisation.

5. Ethical Considerations

Fair play. A tool that instantly suggests the best move could be misused for cheating. VICA is intended strictly for post-game analysis, training, and study; using engine assistance during rated games violates the rules of all major chess platforms and federations.

Dataset bias. The classifier was trained on a specific piece style and is reliable only on similar boards. Deployed without awareness of this limitation, it could produce confidently incorrect results — a representational bias rooted in non-diverse training data.

Privacy. Board screenshots contain no sensitive data, but those from online platforms may include usernames or ratings. VICA processes all images locally and does not transmit or store them.

6. Conclusion

VICA demonstrates a complete computer vision pipeline that combines classical image processing, deep learning, and symbolic rule-based reasoning to convert a chess board image into an engine-backed best-move recommendation, accessible through both a command-line and a web interface. The system meets its goals on in-domain inputs and handles its limitations transparently. Future work includes a larger and more diverse training dataset, robust automatic board-corner detection, and support for photographs of physical boards.